



Habib University

EE-371/CS-330 Computer Architecture – Fall 2025

Instructor: Dr. Shehzad Hasan

Time = 60 minutes

Quiz 02

Max Points: 20

Instructions:

- i. **Smart watches, laptops, and similar electronics are strictly NOT allowed.**
- ii. **Answer sheets should contain all steps, working, explanations, and assumptions.**
- iii. Attempt the quiz on clean papers with black/blue ink.
- iv. Print your name and HU ID on all sheets.
- v. Turn in your question paper along with your answer sheets.
- vi. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	✓
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	
CLO 4	<i>Analyze the</i> performance of cache operations	

Question 1 [5 points]:

Compile the following C expression into RISC-V assembly language. Please archive all the necessary registers onto stack.

```
long long int summation (long long int elements[], size_t n) {
    size_t i = 0;
    long long int sum = 0;
    while (i < n) {
        sum = sum + elements[i];
        i++;
    }
    return sum;
}
```

Since no saved register is used and also since we are not calling another function from summation there is no need for stack memory in this procedure

```
label:  li t0, 0           # index i
        li t1, 0           # sum
again:  bge t0, a1, end     # branch if i is greater than or equal to n
        slli t2, t0, 3      # i*3 multiply by 8 because of long long int
        add t2, a0, t2      # addr = base + i*3
        ld t3, 0(t2)        # get value of elements[i] in t3
        add t1, t1, t3      # sum = sum + elements[i]
        addi t0, t0, 1      # i++
        j again
end:    mv a0, t1           # put value back in a0 before returning
        ret
```

Question 2 [5 points]:

Use a table similar to what is shown below, calculate the product of 5-bit unsigned integers 12 (01100) and 20 (10100). You should show the contents of each register on each step.

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 <u>1</u>	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 <u>1</u>	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial Values	0110 <u>0</u>	00000 10100	00000 00000
1	1: $0 \Rightarrow$ No Operation	01100	00000 10100	00000 00000
	2: Shift Left Multiplicand		00001 01000	
	3: Shift right Multiplier	0011 <u>0</u>		
2	1: $0 \Rightarrow$ No Operation	00110	00001 01000	00000 00000
	2: Shift Left Multiplicand		00010 10000	
	3: Shift right Multiplier	0001 <u>1</u>		
3	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	00011	00010 10000	00010 10000
	2: Shift Left Multiplicand		00101 00000	
	3: Shift right Multiplier	0000 <u>1</u>		
4	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	00001	00101 00000	00111 10000
	2: Shift Left Multiplicand		01010 00000	
	3: Shift right Multiplier	0000 <u>0</u>		
5	1: $0 \Rightarrow$ No Operation	00000	01010 00000	00111 10000
	2: Shift Left Multiplicand		10100 00000	
	3: Shift right Multiplier	00000		

Product = 0011110000 = 240

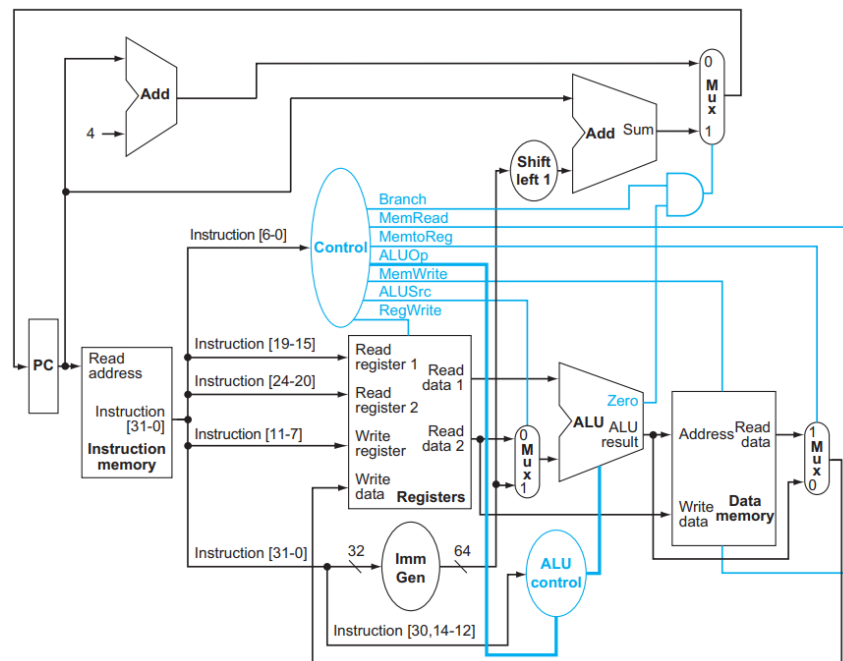
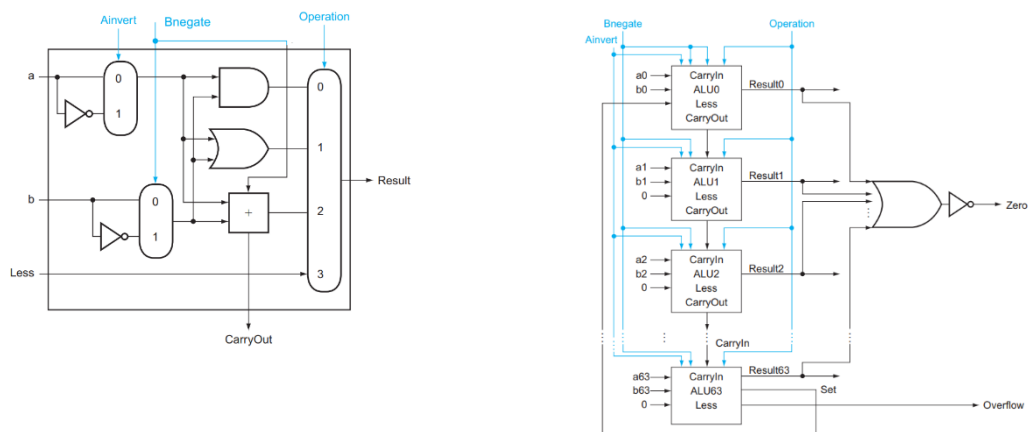


Figure 1: The single cycle datapath with control unit

Question 3 [5 points]:

For the datapath shown in Figure 1, what new datapath elements or control signals are required if another instruction bltu (branch if less than unsigned) is also supported. Draw the necessary changes required in the single cycle datapath design. (Remember we can get the 'less than' signal from the



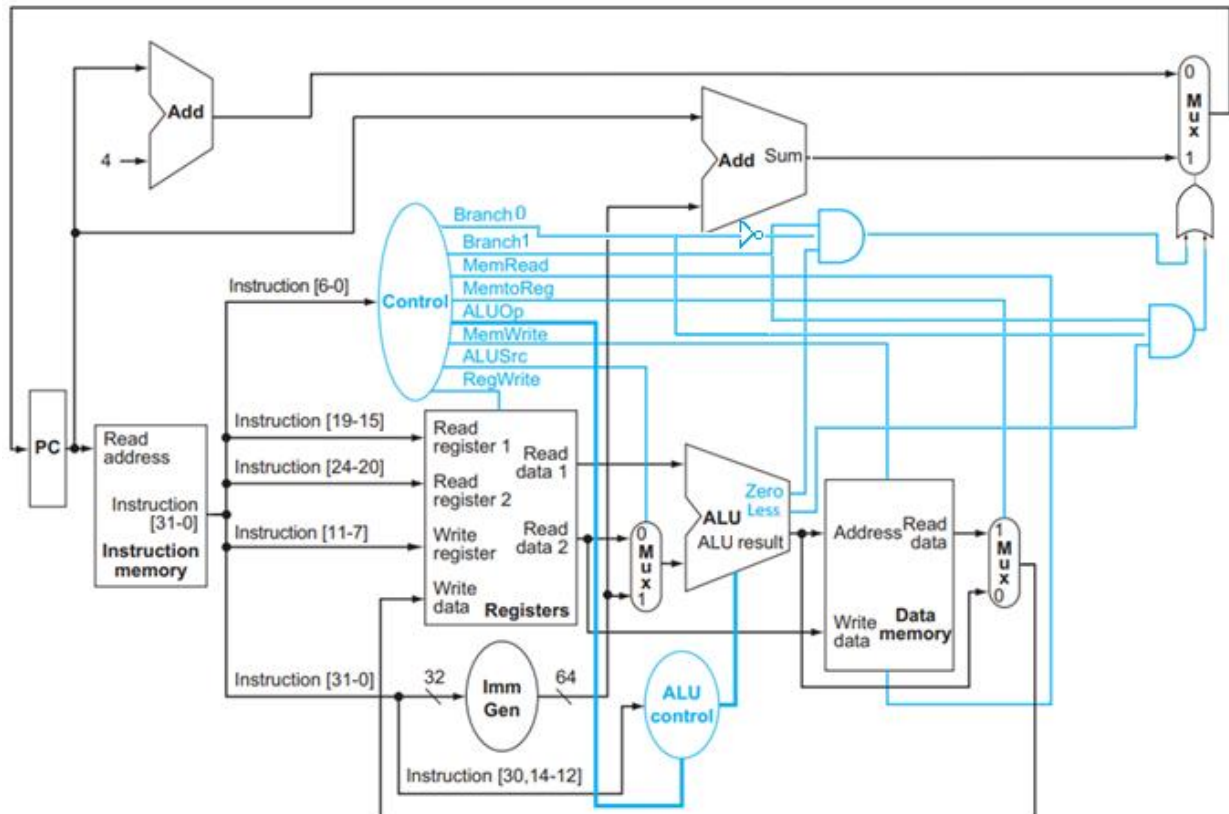
ALU)

The control unit has to differentiate between beq and bltu operations. Since all branch instructions are same the funct3 bits should also be provided to control unit to generate two signals for branch.

Branch1	Branch0	Comment
0	0	No branch
1	0	Beq
1	1	Bltu

Secondly the ALUOp signal has to change as we are not interested in a simple subtraction operation as done in beq instruction but a less than operation. We could use ALUOp as 11 to specify less than operation which would use the less output. In the question it is mentioned that the ALU is capable to generate such a signal so the only thing now left is to create the necessary PCSrc logic involving both beq and bltu instructions.

Looking at the codes we have assigned branch1 can differentiate whether it's a branch instruction or not, while branch0 can differentiate the two branches 0 for beq and 1 for bltu.



Question 4 [5 points]:

For the instructions given below, identify the datapath elements used by each instruction and the value of each control signal (ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp1, ALUOp0)

Instruction	Datapath elements used	Control signals
<code>add x2, x10, x11</code>	Instruction Memory Register File (Read and Write) ALU	ALUSrc = 0, MemtoReg = 0, RegWrite = 1, MemRead = 0, MemWrite = 0, Branch = 0, ALUOp1 = 1, ALUOp0 = 0
<code>ld x3, 64(x2)</code>	Instruction Memory Register File (Read and Write) ALU Data Memory	ALUSrc = 1, MemtoReg = 1, RegWrite = 1, MemRead = 1, MemWrite = 0, Branch = 0, ALUOp1 = 0, ALUOp0 = 0